

tf.name_scope Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/name_scope

A context manager for use when defining a Python op.

Function Signatures

```
tf.name_scope( name ) -> None  
  
def my_op(a, b, c, name=None): with tf.name_scope("MyOp") as  
scope: a = tf.convert_to_tensor(a, name="a") b =  
tf.convert_to_tensor(b, name="b") c = tf.convert_to_tensor(c,  
name="c") # Define some computation that uses `a`, `b`, and  
`c`. return foo_op(..., name=scope)
```

Code Examples

```
tf.name_scope(  
name  
) -> None
```

```
tf.name_scope(  
name  
) -> None
```

```
def my_op(a, b, c, name=None):
    with tf.name_scope("MyOp") as scope:
        a = tf.convert_to_tensor(a, name="a")
        b = tf.convert_to_tensor(b, name="b")
        c = tf.convert_to_tensor(c, name="c")
        # Define some computation that uses `a`, `b`, and `c`.
    return foo_op(..., name=scope)
```

```
def my_op(a, b, c, name=None):
    with tf.name_scope("MyOp") as scope:
        a = tf.convert_to_tensor(a, name="a")
        b = tf.convert_to_tensor(b, name="b")
        c = tf.convert_to_tensor(c, name="c")
        # Define some computation that uses `a`, `b`, and `c`.
    return foo_op(..., name=scope)
```

```
__enter__() -> str
```

```
__enter__() -> str
```

```
__exit__(
    type_arg: None, value_arg: None, traceback_arg: None
) -> bool
```

```
__exit__(
    type_arg: None, value_arg: None, traceback_arg: None
) -> bool
```

Documentation

A context manager for use when defining a Python op.

Used in the notebooks

- Migrating model checkpoints
 - Displaying text data in TensorBoard
 - Graph-based Neural Structured Learning in TFX

This context manager pushes a name scope, which will make the name of all operations added within it have a prefix.

For example, to define a new Python op called `my_op`:

When executed, the Tensors `a`, `b`, `c`, will have names `MyOp/a`, `MyOp/b`, and `MyOp/c`.

Inside a `tf.function`, if the scope name already exists, the name will be made unique by appending `_n`. For example, calling `my_op` the second time will generate `MyOp_1/a`, etc.

Raises

`## Attributes`

Methods

`## __enter__`

View source

Start the scope block.

`__exit__`

[View source](#)

Raise any exception triggered within the runtime context.